CASE FILE · TECHNICAL REFERENCE

ENGAGEMENT WORKFLOW v0.33.6

End-to-end flow from request to closed engagement · compliance surveillance engineering team

New here? Start with the one-page quick-start reference; this sheet is the technical companion to it. The dark panels are **real captured output** from a recorded run (full transcript), placed against the phase they evidence.

STATE

⏳ In progress
⛔ Blocked
✅ Closed

EVIDENCE TAGS

📊 Observed / measured
🧠 Inferred
📄 Coded

WHAT THIS SHOWS

How an AI delivery team inside Claude Code takes a piece of work from first request to a closed, signed-off delivery. Six phases, read top to bottom.

WHO'S WHO

Morgan is the AI project manager and your single point of contact. The specialists are AI agents with fixed roles and names. You are the human; every gate waits for your answer.

HOW TO READ IT

Follow the arrows downward. ◇ diamond boxes are decisions, 👤 orange boxes are actions only you can take, red boxes are hard stops. Pills on arrows are the branch taken.

0 REQUEST & OPEN

Wake the team only when asked, then find out what the work is about.

👤 User request arrives

↓

◇ Team invoked?

- `/engage` or another team command
- or asking in words for "the team" / "Morgan" / "the PM"

NO

Plain Claude Code session

Everything below stays inert. Only the Layer 0 guards stay armed (see sidebar).

YES ↓

Step 0: one compound probe

- Single Bash call, no narration turns
- Loads: operating guide · Python · run mode · version
- Plus: analyser inventory · codebase-map header + history · changelog

↓

Opening banner

- 🎩 Morgan intro + team version + `/meet-the-team` offer
- What's-new line only if the version changed since last visit

↓

◇ Concrete target or inputs given?

NO (BARE /ENGAGE)

👤 Ask ONE question, then wait

- "Where is the code / input?"
- Never invent a target; the safety questions below need one first

YES ↓ Phase 1

RECORDED RUN · WHAT YOU ACTUALLY SEE SHOW ▾

WHY THIS PHASE EXISTS

This phase exists so the team stays invisible until asked. Dormant by default is a cost decision and a safety one: nothing loads, nothing runs, nothing reads.

1 SAFETY & CONSENT GATES

Two safety questions before any work starts. Asked once, never re-asked.

Two safety notices, shown word-for-word

- **Execution safety:**
  nothing is run by default; the user owns the risk of any code they clear for running
- **Data safety:**
  everything shared goes to the model provider; `data/raw/` is hard-blocked; pseudonymised ≠ anonymous

↓

One batched question call

- Only the questions that apply: work type · execution consent · data attestation

↓

◇ Execution consent?

Default No. A "yes" here is only intent; the grant is a human act, below.

YES, WILL GRANT

👤 The human creates the marker

- User personally types `touch .claude/.exec-consent`

- The model is blocked from writing it
- Marker verified to exist before anything runs; no marker = no execution

NO, STATIC ONLY

Any existing marker deleted

- Fail-safe close
- Dynamic and performance findings stay 🧠 inferred

↓

◇ **Data attestation: is the data safe to share?**

NO / UNSURE

👤 `/prepare-data` **first** ALPHA

- 👤 **Do not rely on it to anonymise.**
  Alpha, best-effort, not an anonymisation pipeline: bring data already masked by approved tooling
- Best use: generate fully synthetic data
- Masking is a rough pass (schema + validation, key from `MASKING_KEY`) that you still verify; masked output remains personal data

CLEAN, OR NO DATA

Continue

Attestation recorded; the responsibility is the user's.

RECORDED RUN · CONSENT IS HANDED BACK TO YOU SHOW ▾

WHY THIS PHASE EXISTS

This phase exists because consent cannot be inferred. Intent stated in chat is not a grant; only a human touching the marker is.

2 CLASSIFY THE WORK

Decide what kind of work this is; each kind takes a different path to the same gate.

◇ **What kind of work?**

A deliverable can be any surveillance-engineering output: rule, pipeline, script, reporting job, tooling, review. Route by type, never assume detection rule.

PROBLEM / IDEA

Requirements first

- Business requirements: `/write-brd` · `/elicit-requirements`
- Then a functional spec: `/brd-to-fsd`
- Then build, through the full delivery lifecycle

REVIEW

◇ **Security-sensitive surface?**

- Auth, parsing, DB, external I/O, crypto, secrets, PII

- Yes → offer review + `/security-audit` (recommended); re-offered at review close

↓

Review menu, locked wording, one call

- **Depth:**
  Quick, Deep or Audit (each level includes the one before), or None
- **Performance review:**
  yes / no (static, no code run)
- **What happens to findings:**
  report only / apply fixes / fix-and-re-review loop ⟲

DEPTH NONE + PERFORMANCE NO

Nothing to run

Say so; ask what outcome is wanted instead.

BUILD FROM REQUIREMENTS

`/build-solution`

Straight to plan & gate.

Phased engagements re-classify per phase: the moment any phase produces deliverable code, it runs under the full build chain (review + independent QA + the Definition of Done, the "done" checklist used at close), however the engagement started.

WHY THIS PHASE EXISTS

This phase exists because the wrong route is expensive. A review dressed as a build wastes a fan-out; a build dressed as a review ships untested code.

3 PLAN & GATE

Agree what will be produced. Nothing is delivered until the user says proceed.

Artifact menu, locked, two stages

- **Stage 1 packaging:**
  Consolidated Delivery Report (default) / separate artifacts / both
- **Stage 2**
  (only if separate/both): grouped picks - spec docs · reviews · handover
- Every artifact ships as `.md` + `.html`

↓

📄 Engagement Brief + 📄 START-HERE index (⏳)

- Brief: decisions, assumptions, open questions, routing plan
- START-HERE: one file that lists everything produced and what's still outstanding, created in the same breath
- From here on: every artifact write appends a START-HERE row in the same turn

↓

◇ Go-ahead gate

ADJUST

Back to the artifact menu

STOP

Delivery never starts

Brief + index stay on disk.

PROCEED

Phase 4: delivery loop

WHY THIS PHASE EXISTS

This phase exists so cost and scope are agreed before spend, not explained after it.

4

DELIVERY LOOP ⟲

A small named team does the work; the PM challenges it; all code goes through the full chain.

Right-sizing, stated out loud first

- One line naming how many agents and why
- Multi-agent ≈ 15× tokens, so: leanest set that fits
- 1 agent for fact-finding · 2-4 for comparisons · minimal chain for delivery

↓

Route to named specialists by deliverable

- Morgan + 16: Amara requirements · Mateo rules · Kenji pipelines · Ana analysis · Theo tuning · Mei ML · Viktor validation · Linh QA · Ravi code review · Thabo performance · Layla compliance · Yuki data quality · Hassan, Camila and Cleo domain experts · Pip mechanical scoring
- Advisory roles cannot edit code: they recommend, builders implement
- Model tiers: the strongest model (opus) for last-word judgement · mid (sonnet) for build and advisory · cheapest (haiku) for mechanical scoring

↓

Specialists work on the blackboard

- The blackboard = the shared files: agents coordinate by reading and writing artifacts, not by talking to each other
- Explicit, non-overlapping briefs: objective · scope · inputs · output format
- Replies back to the PM stay short; detail goes to the shared files
- Every critique names an external standard; the critic is never the author

↓

PM challenge: sceptic, not relay

- Challenge every Critical, anything regulated, and thin evidence
- Sample the rest AND the filtered-out set; promote wrongly filtered findings
- Verify evidence tags; inference never reaches the user dressed as fact

↓

If code is produced: mandatory chain, no exemptions

- Tests (project's own framework, exact command recorded)
- → code review ⟲ (fix cycles follow the findings choice made at the review menu)
- → independent QA by Linh ⟲ (cycles append-only, evidence kept on disk)
- → Definition of Done
- Consent withheld? Static-only path: QA verdict stays 🧠, DoD marked PARTIAL, untested code named as residual risk

↓

◇ **Turn ends needing user input?**

YES → BLOCKED PATH

⛔ **Blocked, but resumable**

- START-HERE set ⛔ with the outstanding list: open questions + every gate not yet run
- Turn ends saying plainly "NOT closed"
- No summary email, no delivery report

↓ resume

Same session or cold resume

- User answers → ⛔ flips to ⏳, answer logged, continue ⟲
- New session: reads START-HERE + interim artifacts as the state of record, honours past decisions, works the outstanding list top to bottom

NO, WORK COMPLETE

Phase 5: close sequence

The only path to ✅.

Naming discipline 📄: pre-close outputs use pass-scoped names (review-pass-N, qa-cycle-N, interim-*) and open with an interim banner. `delivery-report.md`, `final-*` and the summary email are close-only names. Iteration history is append-only evidence, never rewritten.

RECORDED RUN · RIGHT-SIZING, THEN A LOOP THAT EARNED ITSELF SHOW ▾

WHY THIS PHASE EXISTS

This phase exists because a reviewer who saw the builder's reasoning is not an independent reviewer.

5 CLOSE SEQUENCE

Eight ordered steps. START-HERE flips to ✅ last of all; sign-off stays human.

✓ **1 Citations gate**

- `check_citations` runs
- Unverified citations ship flagged, with resolved permalinks
- Never blocks the close

✓ **2 Definition-of-Done gate ⟲**

- The mechanical checker `check_artifacts --fix` runs, treated as a fix-list
- Deterministic findings auto-fixed; judgement calls escalated to the user
- Re-run until clean

✓ **3 Reconciliation sweep**

- Every produced or touched doc, including README and docstrings
- One authoritative set of counts everywhere; stale references removed
- QA evidence retained on disk

✓ **4 Codebase map update**

- Durable architecture facts: 📊/🧠 tags, dates, SHA anchors
- History row appended
- Maps the code, never team activity

✓ **5 Render everything**

- Every `.md` artifact gets its `.html` sibling

✓ **6 Delivery report + summary email**

- Report: close-only, consolidated by default
- Email: `.txt`, signed as Morgan, states the token/agent footprint
- Next steps are actions, never a call or meeting

✓ **7 START-HERE finalised last**

- Status → ✅ CLOSED + date; verdict and footprint filled in
- Interim banners stripped
- The flip to ✅ is the final state change

✓ **8 Close with next steps**

- Short summary + concrete options with a recommendation
- A dead end is a PM failure
- Human sign-off is the one item only the user can perform

RECORDED RUN · THE CLOSE STATES THE JOURNEY, NOT JUST THE VERDICT SHOW ▾

WHY THIS PHASE EXISTS

This phase exists because 'done' claimed is not 'done' evidenced. The gate runs here, and refuses.

LAYER 0 · ALWAYS-ON GUARDS

Every session, even when the team is dormant ▲

PRETOOLUSE · ONE DISPATCHER, THEN THE GUARDS IN ORDER

Permission deny rules

Hard-deny read/write on `data/raw/**`; read deny on `.env*`, `secrets/**`, `*.pem`, `*.key`. Repo-as-project only: a plugin install ships hooks but no deny list, so there the hooks are the sole file-tool control.

`locked_menu_guard.py` · **matcher** `AskUserQuestion`

Keeps the locked question menus (review depth, artifact packaging) to their exact wording and option sets, so a menu cannot silently drift.

`bash_hook_dispatcher.py` · **one process, five checks**

The three safety guards below plus two input redirects used to be five separate hook processes per Bash call. They now run inside one dispatcher, first-block-wins, each keeping its OWN crash policy (safety guards fail CLOSED, redirects fail open).

`guard-raw-data.py`

Blocks Read/Grep/Glob/Bash touching `data/raw/`, plus `WebFetch` on a `file://` URL and any unknown read tool. A search rooted ABOVE the raw directory blocks too, but only when raw records actually exist, so a synthetic-only project keeps normal repo-wide search.

`guard-code-execution.py`

Bash only. Team scripts run consent-free (`convert_file`, `render_html`, `check_artifacts`, `engagement_state`, `engage_probe`, `validate_rtm`, `validate_references`, `eval_score`, ...). Executing anything else needs the human-made `.claude/.exec-consent` marker or `CST_ALLOW_EXEC=1`, else it blocks. Static by default; consent is a human act.

`guard-consent-writes.py`

Blocks every model write to the consent marker, settings, hook files and `.git/config`. The model can never grant itself consent, weaken a guard, or reach the same effect by pointing git's hooks path at code of its own.

Input redirects

`document_input_redirect.py` forces PDFs/DOCX/spreadsheets through `convert_file` instead of being hand-parsed; `module_form_redirect.py` normalises how team scripts are invoked.

→ Tool runs

`persona_anchor.py` **· every user turn**

While an engagement is live: re-inject a ~220-token Morgan persona and discipline anchor, so the team survives even when the chat history gets condensed. It points at the roster rather than inlining it. Otherwise silent.

`post_edit_lint.py` **· after every write**

Lints Python the moment a builder writes it (syntax always, ruff when installed) and feeds problems straight back, instead of waiting for the review gate.

`subagent_return_budget.py` **· after a specialist returns**

Advisory nudge when a subagent's reply is clearly over the ~1,500-token return budget, since an over-long return lands verbatim in the PM's context.

`session_resume_brief.py` **· on compact / resume**

Re-briefs a resumed or compacted session: which engagement is active, and that answers already given live on disk and must not be re-asked (a consent "No" stays "No").

`dod_stop_gate.py` **· when a turn tries to end**

If an engagement is open: run the mechanical DoD check. Clean → silent. Defects → warn once with a fix-list: auto-fix the deterministic ones, escalate judgement calls, or end plainly saying NOT closed.

`todo_panel_nudge.py` **· when a turn tries to end**

Reminds once if a delivery-phase engagement never seeded the native task list, a rule a live audit found was going unfollowed.

SHARED MEMORY & COORDINATION ▲

The BLACKBOARD

Specialists coordinate by reading and writing shared artifacts (the spec, the traceability matrix, the report), not by chatting. Each step's output is the next one's input.

The ENGAGEMENT STATE source of truth

Each engagement owns a workspace `artifacts/<slug>/` holding `engagement-state.json`: status, phase, outstanding work, artifact inventory, decisions, gate answers, footprint. Mutated only through `scripts.engagement_state`, every mutator re-validating and re-rendering in the same command. Several engagements can coexist; one is active per session, recorded on disk. Execution consent is deliberately NOT representable here: it lives only in the human-made marker.

START-HERE rendered, never hand-edited

The human-readable index, generated FROM the state with a content hash, so a hand-edit is itself a detected defect. It survives compaction, and is what a resumed session reads to pick up where it left off.

The CODEBASE MAP

The project's durable memory of its own code: architecture facts with dates and tags. Read at every open, updated at every close. Never a log of team activity.

LAYER 1 · CHECKS BEFORE ANYTHING IS COMMITTED

Contributor machine, then CI. Nothing to do with a live engagement ▲

Local, on `git commit` pre-commit

gitleaks, a raw-data file block, whitespace and private-key checks, and `pytest` when `rules/`, `tests/` or `scripts/` changed. Optional but recommended: it is the layer in front of CI, not a replacement for it.

CI · every push to `main` and every PR

Four jobs in parallel; the test job fans out to three legs (Python 3.10 and 3.12 on ubuntu, 3.12 on windows-latest, because the guards and launcher have Windows-specific paths).

Tests

The full `pytest` suite: rules, masking, the renderer, all three safety guards driven through their real JSON protocol, hook-config sync, the per-case eval contract, and the DoD artifact checker.

Content validators

`validate_masking` (no original PII survives, and detection still fires on masked data) · `validate_manifest` (every declared agent, skill and hook resolves) · `validate_references` (below).

Lint & security

`ruff check` + `ruff format --check` · `bandit` (Python security) · `shellcheck` (Bash and the git hooks) · `gitleaks` over the FULL history, not just the diff · a job that fails if any data file is tracked.

`validate_references.py` the reference walker

A link checker pointed INWARDS. It walks the docs and prompt surface, extracts every path a document, skill or agent mentions, and fails on any that will not resolve. The repo's most common defect is a document citing something that moved or was renamed, and each instance used to be caught only by adding another bespoke assertion after it broke. Deliberately ignores three classes so it does not cry wolf: files the team creates at RUNTIME or that live in the USER's project, placeholders and globs, and a short known-absent list where every entry carries a written reason. `--orphans` also lists docs nothing references.

What CI deliberately cannot cover

Engagement deliverables (`artifacts/` is git-ignored; `check_artifacts` is the gate for those) and live team quality (that needs the eval harness below, which spends tokens). CI proves the harness CONTRACT, not the team's output.

LAYER 2 · THE EVAL HARNESS & RELEASE GATE

Does the team still behave? Milestone-only, spends real tokens ▲

Golden cases 43, in `evals/cases/`

Each carries a scenario or input the agent sees, plus an `expected.yaml` manifest naming the behaviours that must be surfaced (`planted`) and the false-positive traps that must NOT fire (`forbidden`). The answer key lives in `notes.md` , never in the agent-visible input, and a test enforces that.

`eval_engage.py` · the driver

Runs a REAL headless `/engage` session per case through the Agent SDK, in a throwaway sandbox with `evals/` excluded so ground truth is structurally unreachable. The guard hooks stay live: they are under test too. An LLM stakeholder answers the gates. Wall clock is per-case, declared in the manifest.

`eval_score.py` · deterministic scorer

The regression backbone, and unit-tested. Matches the team's evidence against the manifest: recall on planted behaviours, must-find items, traps triggered. Evidence is what the team actually DID: artifacts it wrote, questions it asked, its own prose. Seeded fixtures are excluded, and a promise ("I'll fix that next") cannot satisfy a spec that asserts the work happened.

LLM judge qualitative only

Scores rubric dimensions the deterministic layer cannot see, against a per-case rubric in `evals/rubrics/` , needing ≥ 0.75 and no auto-fail. One uncalibrated call, so it is a coarse signal: recall is the load-bearing number.

Outcome & trend

Every run is classified
pass / fail / unscorable
, so a session killed by a timeout or an API error is never counted as the team answering badly. Rows append to the tracked `evals/results.jsonl` , making scores trendable across releases instead of re-narrated in prose.

`release_gate.py` · promotion to `main`

Checks version, badge and CHANGELOG agree; a baseline exists for the version; no prompt file was committed after it AND none is dirty in the working tree; and the baseline's machine-readable verdict block parses, reconciles arithmetically, covers enough distinct cases, reports zero unadjudicated failures, and

matches the runs recorded in the results log. Adjudicating a failure to a pass is allowed but must be COUNTED, never absorbed into prose.

LEGEND ▲

- Process step (colour = phase)
- ◇ Decision
- 👤 User action / input
- 📄 Output / artifact
- Blocked / halted
- Terminal / outside the flow
- ⟲ loop · pill labels on arrows are branch conditions

Source of truth: `docs/internal/engagement-flow-spec.md` (v0.33.6) and `docs/internal/engagement-flow-diagram.md`. Edit this file and re-render; content is reconciled to the spec, not to any generated image.